

Operating System

CPU registers

Cache

Main memory (Permanent memory)

Hard disk (Secondary memory)

Optical devices

External devices

1. Memory mgt.

2. Processes mgt.

3. Resource allocation

Uses of OS:

1. Program execution

2. File system mgt.

3. Device mgt.

4. Communication (Networking)

5. Errors Handling

6. Accounting (CPU utilisation, memory usage)

7. Security (unauthorised access)

Types of systems:

1. Parallel systems : no dependencies ; helps to ↑ speed

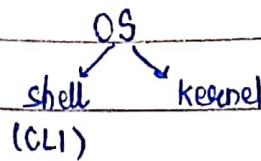
2. Distributed systems : high speed, secure, reliable

Each processor has its own local memory.

Slower than parallel systems.

3. Real-time systems : (i) Hard real time - eg. self driven cars, aviation softwares

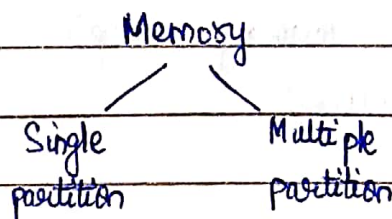
2. Soft real time - eg. ticket booking



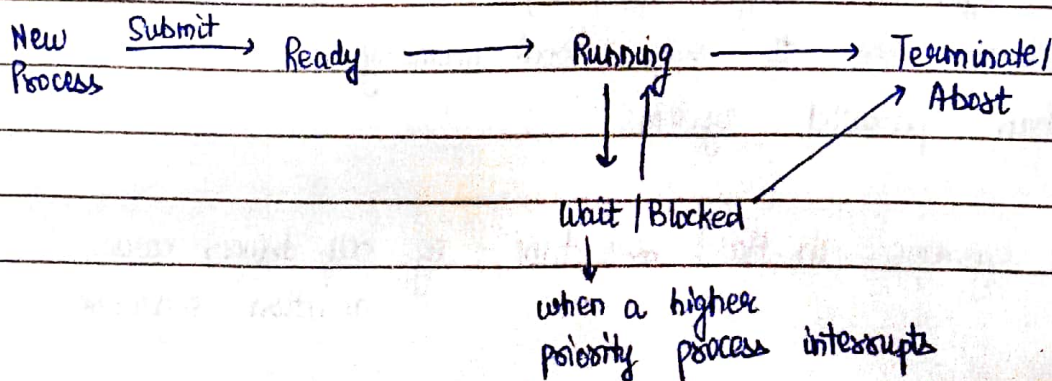
System calls - calls related to hardware

1. Process
2. File System
3. Communication - sockets
4. Errors Handling - device, i/o
5. Security
6. Information maintenance - eg. updation of system time/date

Relocatable register provides the base address.



Limit register stores the last address where the code is loaded in main memory (to avoid overwriting).



1. 3 main purposes of OS
 2. diff. b/w OS of mainframe & personal comp.
 3. main difficulty that a programmer must overcome for writing OS in real time env.
 4. How does the diff. b/w kernel and user mode form as a rudimentary form of protection system?
 5. Purpose of cmd interpreter. Why is it separate from kernel?
 6. Purpose of system calls
 7. 5 major acts of OS in process mgt
 8. main adv. of micro control design for system design
 9. User level thread and kernel thread
 10. Design of a system to allow it to boot. Bootstrap program needed for it.
 1. → CPU scheduling
 - Memory allocation
 - Processors mgt.
2. Mainframes have concurrent users so there are security constraints. They are also less reliable.
3. Time constraint is the biggest problem. Such systems are hard real time systems. The output produced should occur within a specific time interval.
4. Kernel mode lets us interact with the hardware and the internal architecture. It has authorization limits. User mode only lets us interact with the applications through either cmd or gui.
5. Cmd interpreter lets us interact with various applications by using commands rather than a gui.

unsub'd
24/11

The cmd interpreter decodes the command and then further execution takes place.

Kernel is the internal architecture of an operating system. It can't be accessed easily by a user using the command line as the user may willingly or unwillingly tamper with the internal architecture of the operating system.

Scheduler

Compilation → .obj file created → Load into main memory → Linkers

Schedulers determine the processes that are ready to run and the order in which they will run.

1. Long term scheduler: identify all ready processes and put them in job queue
2. Short term scheduler/
CPU scheduler: pass a process from job to ready queue in the order in which it will be run
3. Medium term scheduler: when a process goes into waiting state

Process Control Block (PCB)

Pointer Process	Pc ID
Process list of	state open files
Registers	

logical address - generated by CPU

physical address - generated by our code

Relocatable seg. stores the base addrs. of the code.

$$\text{Physical address} = \text{Relocatable} + (\text{logical addrs.})$$

A process running in kernel mode can directly access the hardware while one running in user mode can't.

1. Independent processes - processes independent of each other
2. Cooperative processes - processes having some shared interests
eg. playing music from 2 applications

Q.

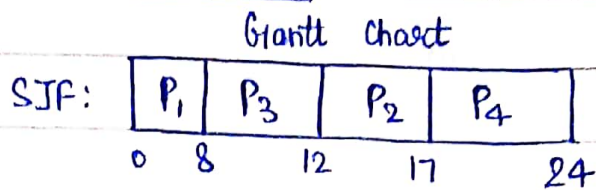
	Arrival time	Burst time
P ₁	0	8
P ₂	1	5
P ₃	2	4
P ₄	3	7

Sol.ⁿ FCFS :

	Waiting time	Turnaround time (W+B)
P ₁	0	8
P ₂	7	12
P ₃	11	15
P ₄	14	21
Avg. WT = $\frac{32}{4} = 8$		Avg. TAT = 14

Turnaround time = completion time - arrival time

SJF (Shortest Job First):



	WT
P_1	0
P_2	11
P_3	6
P_4	14

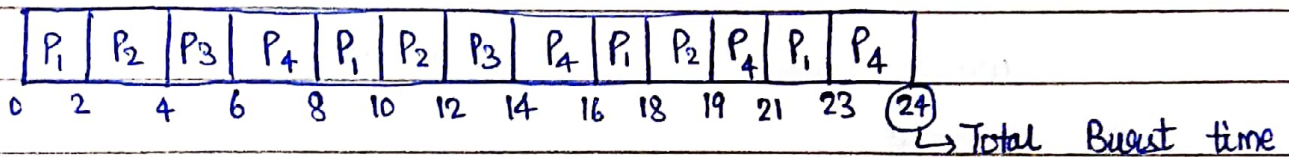
Shortest processes are executed first.

$7.75 < \text{Avg. WT of FCFS}$

Disadvantage of SJF is that whole queue needs to be scanned.

Round Robin:

Irrespective of the burst time, same CPU time slice is allotted to each process.



	WT	TAT
P_1	$0+6+6+3=15$	23
P_2	$1+6+6=13$	18
P_3	$2+6=8$	12
P_4	$3+6+3+2=14$	21
	$\text{Avg.} = \frac{50}{4} = 12.5$	$\frac{74}{4} = 18.5$

This method ensures time sharing and multi-tasking.

ASSIGNMENT 2

1. System call is interaction b/w applications and kernel of OS.

→ File system

→ Process management

→ Communication

→ Device mgt. and security

3. 1. Program execution

2. File mgt.

3. Error Handling

4. Security

5. Accountability

4. Microkernel is faster in execution.

5. Partitioning is done to allow a choice of OS to boot from.

Bootloader loads part of kernel needed for further execution, in main memory.

Virtual m/c can also be used for partitioning.

6. User-level

Kernel-level

→ lightweight

→ user level application

→ easy and simple

→ blocked state of a thread
blocks entire program
execution

→ complex

→ kernel level application

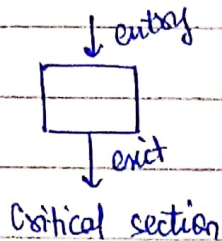
→ direct access to resources

→ new thread replaces a
blocked thread

Process Synchronisation

2 processes can't access the critical section simultaneously.

Cooperative processes are the ones which access the same resource.



Entry in critical section is in serial manner.

Disadvantage is that all the processes are constantly running a while loop to check if critical section is unoccupied. This wastes no. of CPU cycles.

To overcome this disadvantage, semaphore (wait-and-signal) is used.

Dining Philosopher's problem → Process synchronization + Resource allocation

Memory allocation:

size of program = 140

	Unused	
	200	First fit
	150	Best fit
	300	Worst fit

Worst fit case is advantageous as more memory is left thus allowing contiguous memory allocation. This also eliminates external fragmentation, where another program is stored somewhere else and not in contiguous manner.

Internal fragmentation arises due to the space left in a block after a program has been loaded in it. There is no solution for this problem.

Page Table: a data structure which maintains a record of the pages loaded in main memory.

1. Direct mapping:

11	
10	
01	
00	

	000
	001
	010
	100
	101
	110
	111

→ Advantage is low searching time.

→ Disadvantage is that despite empty locations in main memory, pages

from sec. memory are mapped to specific locations.

Page no. / no. of pages

(000) → 0%4 → mapped to location 00 in main memory

2. Set associative mapping:

Page no. / no. of sets

Demand Paging - fetching only demanded pages by CPU from secondary memory.

Thrashing - continuously reloading pages from secondary memory.

This results in overhead as it leads to time waste.

The problem is solved by page replacement algorithms.

1. FIFO

2. Least currently used

3. Optimal algorithm

W/amp
14/12

ASSIGNMENT-3

1.) n no. of processes

$$\text{No. of possible schedules} = n \times n-1 \times n-2 \times \dots \times 1 \\ = n!$$

2.)

Preemptive

1. Resources are allocated to a process for a limited time.

2. Process can be interrupted in between.

3. Low priority process may starve if a high priority process arrives.

4. Flexible

Non Preemptive

Once resources are allocated to a process, it holds it until completed or switched to waiting state.

Can't be terminated unless complete or switched to waiting state

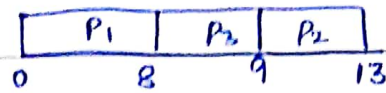
A process with less CPU burst time may starve due to one with more CPU burst time.

Rigid

3.)	Process	Arrival time	Burst time	Waiting time	T.A. time
a.	P_1	0	8	0	8
	P_2	0.4	4	7.6	11.6
	P_3	1.0	1	11	12

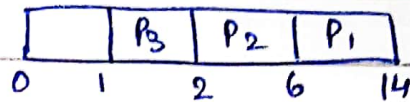
$$\text{avg. T.A. time} = \frac{8+11.6+12}{3} = \frac{31.6}{3} = 10.53$$

b. SJF :



	WT	TA
P ₁	0	8
P ₂	8.6	12.6
P ₃	<u>1</u>	<u>8</u>
avg.	5.2	9.53

c.



	WT	TA
P ₁	6	14
P ₂	1.6	5.6
P ₃	<u>0</u>	<u>1</u>
avg.	2.53	6.86